

Af Marie van Weezel 3r | 7. april 2026

Antal tegn: 14.347



EKSAME PROJEKT

Programmering B | Mikkel Kirkeby Mosthaf

SLOTSHAVEN GYMNASIUM

EKSAME PROJEKT: SPACEMATH

Multispiller matematik spil til læring af små børn

Abstract

Eksamensprojekt er udarbejdet af Jessica Tornqvist og Marie van Weezel og handler om udviklingen af SpaceMath, et interaktivt matematikspil som er målrettet børn i alderen 5- 9 år. Spillet er designet som et multiplayer læringsspil, hvor elever kan løse matematikopgaver for at styre en raket gennem et rummet.

Projektet er opbygget efter tre lag som består af data, logik og præsentations laget. I denne synopsis er der særligt fokus på præsentationslaget, som har været den primære arbejdsområde. Det handler om hvordan brugergrænsefladen er implementeret i Pygame samt hvordan brugerinput håndteres og hvordan spillets forskellige states styrer visningen af de enkelte skærme. Derudover er der kort om designet, hvor der er lagt vægt på en enkel og intuitiv brugeroplevelse med tydelige visuelle elementer og farver så det passer til målgruppen.

Samlet set viser projektet hvordan et interaktivt lærings spil kan struktureres og udvikles med fokus på brugeroplevelse, funktionalitet og en klar opdeling mellem systemets lag.

Link: <https://github.com/neg9414/Eksame-projekt--SpaceMath/tree/main>

Indholdsfortegnelse

Abstract.....	1
Opgave valg	3
Koncept.....	3
Planingen og krav	5
Gruppe fordeling	6
Produkt.....	7
Program.....	7
Design	10
Test	11
Refleksion.....	13
AI.....	13
Konklusion.....	14
Bilag:	15

Opgave valg

Tema 2. Python-applikation med eget tema

I skal udvikle en python-applikation som demonstrerer brug af struktureret brug af trelagsmodellen. Det vil sige at løsningen skal indeholde et data-, et logik- og et præsenteringslag. Datagrundlaget og præsenteringslaget er valgfrit - I bestemmer altså selv hvad løsningen handler om og hvordan den præsenteres for brugeren.

Kraven er del op i to. De "non-funktionelle krav" som handler om at opgaven skal bruge en datakilde og være opdelt i tre lag (data, logik og præsentation), hvor logikken bearbejder data. Der skal anvendes mindst en klasse, AI-instruktioner, samt en velstruktureret og dokumenteret kodebase i et offentligt repository. Den anden er "funktionelle krav" hvor løsningen skal være interaktiv, skulle kunne vise data overskueligt for brugeren, og sikre at brugerinput bliver valideret.

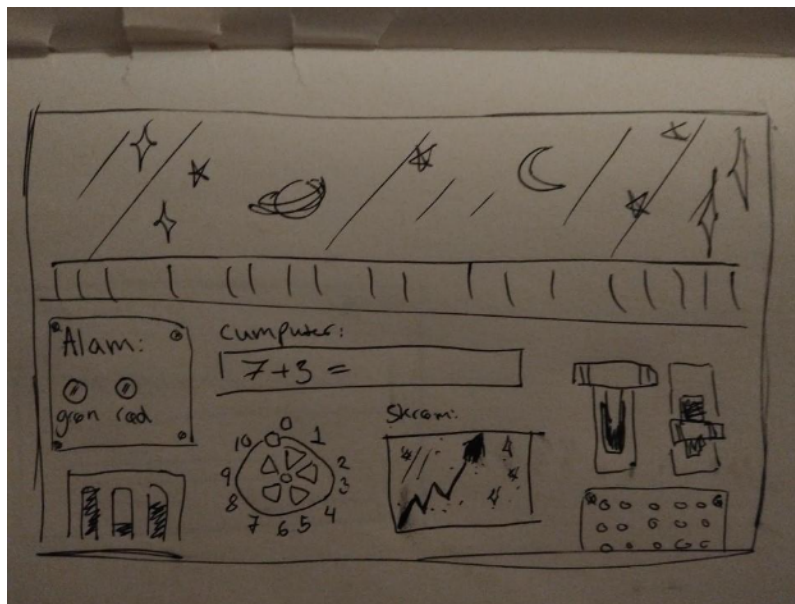
Koncept

Ideen er at lave et interaktiv hjemmeside/spil til læring som er målrettet for elever i alderen 5-9 år. Selve spillet er designet som et 2D rum eventyr hvor elever kan styre en rumraket ved at løse matematikopgaver. For så at få raketten til at flyve frem skal eleven løse regnestykker korrekt for at få nok brændstof.

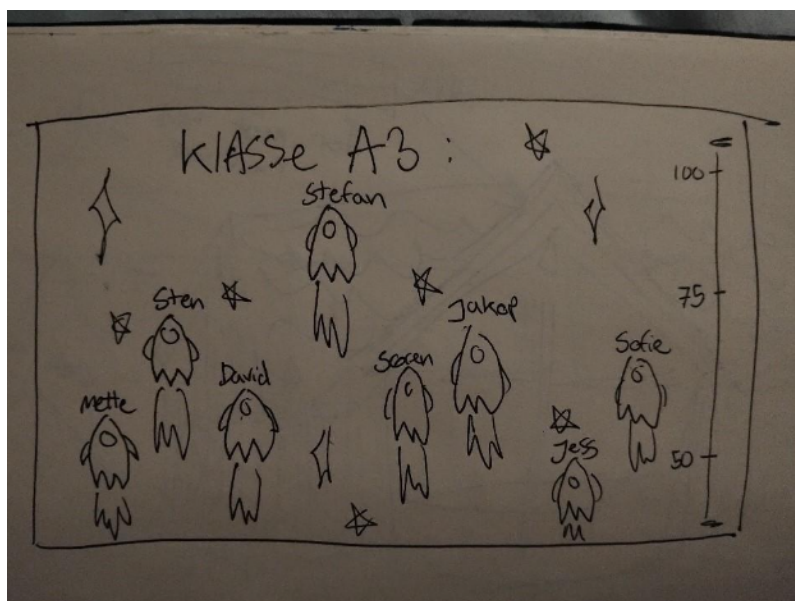
De matematik opgaverne vise med forskellige signaler som grøn lys som betyder at der skal tilføjes brændstof (plus regne stykker) og et rødt lys som betyder at der skal fjernes brændstof (minus regne stykker). Eleven afgiver så deres svar ved at justere et kontrolhjul frem og tilbage.

Det er et multispiller spil hvor flere elever kan deltage i samme spil samtidig. Læreren opretter en session, hvor elever kan deltage via en kode. Eleverne konkurrerer så mod hinanden ved at løse opgaver hurtigst og med mest korrekte. Læreren der opretter spil kan vælge sværhedsgrad, overvåge elevernes progression og se resultater og statistikker over spillet. Derudover har læreren en funktion hvor de kan del deres skærm med stor skærmen eller andet i undervisningen for at se og vise alle elevers raketter flyve mod hinanden.

Her er en visualisering af ideerne:



FIGUR 1 ELEVERNES SKRÆM UNDRER SPILLET



FIGUR 2 LÆRERNES SKRÆM UNDRER SPILLET PÅ EVT. EN STOR SKRÆM

Planingen og krav

For at gøre det mere overskueligt er der lavet milepæle med forskellige krav.

Milepæl 1: One-player game med database (Vigtigt)

En simpel version af spillet hvor kun en spiller kan spille, med matematikopgaver og lagring af data.

Krav:

-  En person skal kunne spille spillet alene.
-  Systemet skal generere matematikopgaver med forskellige sværhedsgrader.
-  Eleven skal kunne afgive svar gennem en interaktiv kontrol.
-  Systemet skal give feedback på rigtige og forkerte svar.
-  Elevens score og progression skal gemmes i et database.
-  Spillet skal udvikles i pygame med simpel grafik og styring.

Milepæl 2: Oversigtsskærm til lærer (vigtigt)

En del af spillet der er egnede kun til læreren, hvor læreren kan se data fra spillet.

Krav:

-  Læreren skal kunne se elevernes resultater og progression.
-  Systemet skal præsentere data struktureret og visuelt.
-  Læreren skal kunne se en samlet oversigt over spillerens status.

Milepæl 3: Multiplayer funktion

Flere elever kan spille samtidig i samme session.

Krav:

-  Læreren skal kunne oprette en spilsession.
-  Systemet skal generere en adgangskode til spillet.
-  Elever skal kunne spille spillet via koden.
-  Elever skal kunne konkurrere mod hinanden i realtid.
-  Læreren skal kunne vise alle spilleres raketter samlet på en skærm.

Milepæl 4 Login system

Brugerhåndtering for at opdele lærere og elever.

Krav:

-  Brugeren skal kunne logge ind som elev eller lærer.
-  Læreren skal have en personlig brugerprofil.
-  Elevers data og progression skal gemmes på deres bruger.
-  Systemet skal håndtere dataforbindelse mellem database og spil.

Gruppe fordeling

Arbejdsfordelingen var lavet på baggrund af gruppemedlemens ambitioner og kompetencer. Så der bliver aftalt at gruppe fordelingen ikke ville være lige fordelt da vi ikke har samme mål til karakter og færdighed.

Jessica har primært haft ansvar for datalaget samt udvikling af lærers grænsefladen, herunder håndtering og strukturering af data.

Marie har haft den overordnede ansvar for systemets arkitektur, herunder design og implementering af præsentationslaget samt udviklingen af den centrale programlogik. Som opbygning af funktionalitet, styring af programmets flow og sikring af sammenhæng mellem de forskellige lag i systemet.

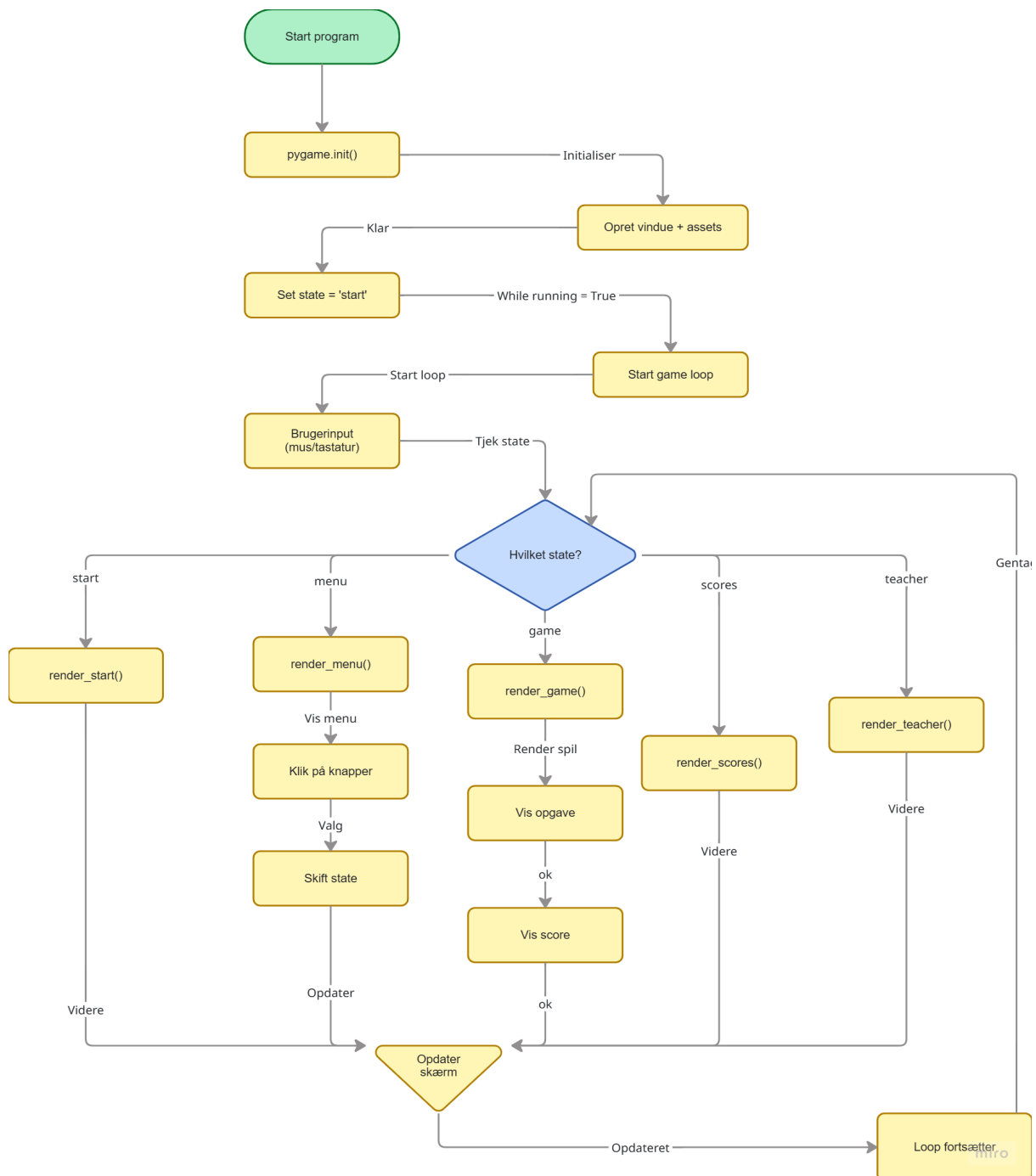
Produkt

Det færdige produkt er et interaktivt matematikspil der er udviklet i Python med Pygame. Spillet er designet til elever i alder 5 til 9 år og har til formål at gøre matematik til et sjovt spil miljø hvor de kan kæmpe mod hinanden og se deres proces. Spillet er bygget som et klassisk game loop, hvor programmet konstant opdaterer skærmen, håndterer input og viser den relevante information til brugeren. Der brugs en state system til at skifte mellem forskellige skærme såsom start, menu, spil og scorevisning. Samlet set opfylder produktet de funktionelle krav ved at være interaktivt, give brugeren feedback og præsentere data på en overskuelig måde.

Program

Selv præsentations laget står for alt det visuelle i programmet og håndterer af brugerens interaktion med spillet. Det er lavet ved hjælp af Pygame og indeholder både rendering af grafik, håndtering af input samt styring af spillets flow. Opdelingen i tre lag gør systemet mere overskueligt og modulært. Det betyder, at ændringer i brugerfladen ikke påvirker selv spillets regler, logik eller database.

Spillet initialiseres med Pygame, hvor vindue, grafik og state system oprettes (*figur 8*). Her initialiseres Pygame og der laves et vindue med en fast størrelse. Der bliver styrter skrifttyper og billeder som bruges i spillets forskellige skærme. Og ud over det bliver der også oprettes en forbindelse til databasen, hvor spillerens data bliver gemt. Spilleren bliver også oprettet i databasen, så deres resultater kan kobles til den unikke bruger. Derudover initialiseres en GameSession, som håndterer spillets logik. På den måde bliver præsentationslaget kun ansvarligt for visning, mens logikken og dataene ligger et andet sted. Til sidst sættes en variabel kaldet "state", som bruges til at styre hvilken skærm spillet er på.



FIGUR 3 RUTEDIAGRAM FOR PRÆSENTATIONSLAGET

Diagrammet viser strukturen i præsentationslaget. Programmet kører i et game loop, hvor brugerinput registreres, og den relevante render funktion kaldes afhængigt af den aktuelle state. Dette gør det muligt at opdele programmet i forskellige skærme og sikrer en overskuelig struktur. Diagrammet viser ved at opdelingen i tre lag, hvordan præsentationslaget fungerer uafhængigt af logik og data.

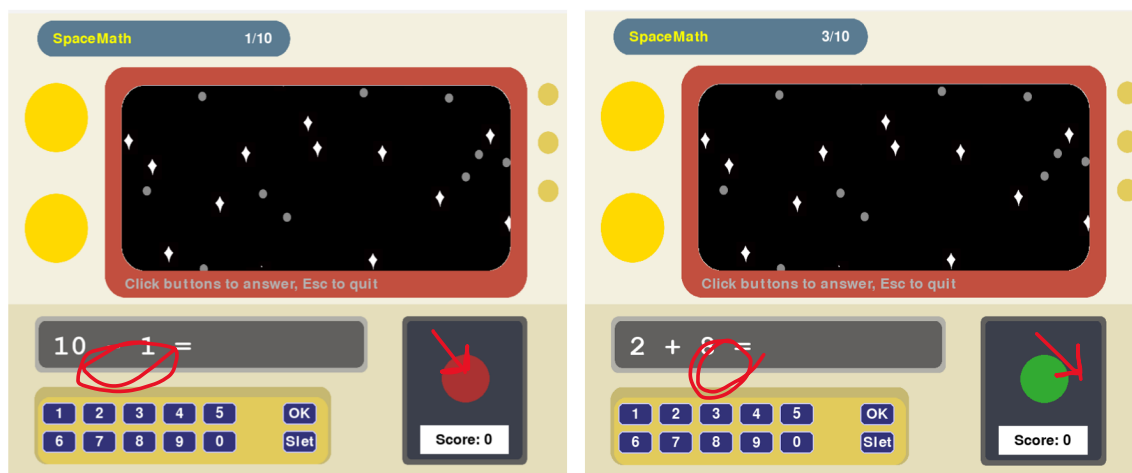
Spillet bruger et state system til at styre hvilke skærme der vises (*figur 8*). State variablen have værdier som "start", "menu", "scores", "competed" og "teacher". Det gør det muligt at opdele programmet i forskellige skærme, som hver har deres egen render funktion.

Game loopet er hjertet i spillet og kører kontinuerligt, så længe spillet er aktivt (*figur 9*). I loopet bliver der konstant lyttet efter brugerinput, såsom tastetryk og museklik. Afhængigt af hvilket state spillet er i, bliver der udført forskellige handlinger. For eksempel håndteres klik forskelligt i menuen og i selve spillet. Efter input er behandlet, bliver den relevante skærm tegnet ved at kalde den tilhørende render-funktion. Render-funktionen står for at tegne selve spillet på skærmen (*figur 10*). Her vises baggrunden, spillerens score samt den aktuelle matematikopgave. Derudover vises enten spillerens input eller feedback, afhængigt af om spilleren lige har svaret. Render-funktionen bliver kaldt mange gange i sekundet gennem game loopet, hvilket gør at spillet føles flydende.

Input registreres via museklik og sendes til logiklaget, som returnerer feedback (*figur 11*). Der er tilføjet preview og typing-effekt for at forbedre brugeroplevelsen (*figur 12, 13*).

Design

Præsentationslaget er designet med fokus på en simpel og visuel brugeroplevelse, da målgruppen er børn i alderen 5-9 år. Derfor er der lagt vægt på tydelige farver, store elementer og minimal tekst, så spillet er let at forstå uden at kræve læsning.



FIGUR 4 FARVEKODNING I SPILLET (GRØN = PLUS, RØD = MINUS)

Farver spiller en vigtig rolle i designet. Grøn og rød bruges til at signalere plus og minus opgaver, hvilket gør det intuitivt for brugeren at forstå opgaven uden forklaring. Dette reducerer kognitiv belastning og gør spillet mere tilgængeligt.



FIGUR 5 INPUT OG FEEDBACK I SPILLET

Brugerinteraktionen er designet gennem klikbare knapper og et simpelt inputsystem, hvor spilleren nemt kan afgive svar. Feedback gives direkte på skærmen, så brugeren hurtigt kan se, om deres svar er korrekt. Dette skaber en motiverende og lærende oplevelse.

Der er også arbejdet med dynamiske elementer som typing effekt og preview funktion, som gør spillet mere levende og engagerende. Disse elementer er ikke nødvendige for funktionaliteten, men hjælper med brugeroplevelsen og fastholder opmærksomheden hos målgruppen. Designet tager udgangspunkt i principper om brugervenlighed og intuitiv interaktion, hvor visuelle signaler reducerer behovet for tekst. Dette er særligt vigtigt for målgruppen, som stadig er i gang med at udvikle læsefærdigheder.

Test

Test af programmet er udført ved at isolere logiklaget fra den grafiske brugerflade og afprøve det direkte i terminalen (*figur 15*). Denne testmetode bruges til at se om at spillets kernefunktioner fungerer uafhængigt af præsentationslaget, hvilket sikrer at der er lavet en rigtig adskillelse mellem logik og UI.

I testen blev der oprettet en instans af `GameSession`, som fungerer som styringsenhed for spillets logik. Så den starter med at generere en matematikopgave direkte fra logiklaget, uden at Pygame eller grafisk rendering er aktivt. Der kan så indtaste svaret ind i terminalen, hvorefter systemet evaluerede svaret, returnerede feedback og opdaterede spillerens score. Derefter blev der automatisk genereret en ny opgave, hvilket simulerer spillets normale flow.

```
=== SpaceMath terminal test ===  
Skriv dit svar og tryk Enter. Skriv 'exit' for at stoppe.  
  
Opgave: 10 - 8 = ?  
Dit svar: █
```

FIGUR 6 OPSTART AF TERMINALTEST

Figuren viser starten på testkørslen af logiklaget. Her initialiseres programmet uden det grafiske, og GameSession bliver oprettet. Spillet kører i et simpelt tekstbaseret miljø, hvor input og output håndteres direkte i terminalen. Det gør det muligt at teste funktionalitet uden at være afhængig af Pygame eller brugergrænsefladen.

<pre>Dit svar: 2 Rigtigt! Score: 1 Løst: 1 Opgave: 2 + 5 = ? Dit svar: █</pre>	<pre>Dit svar: 5 Prøv igen. Score: 1 Løst: 2 Opgave: 0 + 4 = ? Dit svar: █</pre>
---	---

FIGUR 7 FEEDBACK, SCORE OG NYE OPGAVER I TERMINALEN

Figuren viser selve interaktionen mellem bruger og logiklag. Her indtastes et svar til en matematikopgave, hvorefter systemet returnerer feedback baseret på om svaret er korrekt eller forkert. Samtidig opdateres spillerens score og der genereres en ny opgave. Det viser at logiklaget både kan håndtere inputvalidering, tilstandsopdatering og generering af nye data i en kontinuerlig proces.

Testen viser samlet set, at logiklaget fungerer som en selvstændig enhed, hvor inputbehandling, evaluering og dataopdatering sker korrekt uden at være afhængighed af præsentationslaget. Det understøtter lagopdeling og gør det lettere at finde fejl, videreudvikle systemet og generelt mere overskueligt.

Refleksion

Projektet blev i høj grad påvirket af arbejdsfordelingen i gruppen. Selvom opgaven var tænkt som et samarbejde, endte jeg med at få det primære ansvar for udviklingen, herunder både præsentationslaget og den overordnede programlogik og dataen. På baggrund af nogle computerproblemer hos den anden gruppemedlem.

Det har så båret være en fordele og ulemper. På den ene side gav det mig en dybere forståelse for opbygningen af programmet i flere lag og sammenhængen mellem dem. Men på den anden side øget det arbejdspress, hvilket påvirkede meget motivationen og gjorde det sværere at arbejde struktureret og grundigt gennem hele processen. Som gør at projektet kun nåede at fokusere på de to første og vigtigste milepæle, altså kerneproduktet blev færdigt. Så derfor blev nogle af de mere avancerede funktioner, som multiplayer og fuld brugerhåndtering, ikke færdigt. Hvilket er ærgerligt, da der mangler nogle afsluttende finjusteringer, som kunne have skabt en mere sammenhængende og færdig helheds oplevelse i både design og funktionalitet. Næste gang kunne en mere struktureret arbejdsfordeling bidrage til et mere balanceret projektforløb.

AI

I udviklingen af projektet er der anvendt meget AI, herunder Copilot. Brugen af AI blev i høj grad påvirket af det øgede arbejdspress, da jeg i praksis arbejdede alene med projektet.

AI blev anvendt som et værktøj til at generere kodeforslag og hjælpe med at løse konkrete problemer. Det har dog været vigtigt for mig at forstå den genererede kode og tilpasse den til projektets behov. Der har derfor været en løbende dialog mellem mig og AI, hvor jeg har justeret og videreudviklet løsningerne. Samtidig må det erkendes, at brugen af AI i perioder har været omfattende og at største dele kunne være udviklet selv. Det er noget der er tænkt meget over efter i frustrationer. På trods af det har arbejdet med AI bidraget til en bedre forståelse af programmets struktur og opbygning, og det har fungeret som et lærings værktøj frem for blot en løsning.

Konklusion

Projektet har resulteret i et virkene matematikspil udviklet i Python med Pygame, hvor det er tydeligt opdelt i tre dele data, logik og præsentations lag. Hvor opdeling har gjort det muligt at være mere strukturere samt gjort programmet mere overskueligt.

De vigtigste funktionelle krav er opfyldt, da programmet er interaktivt, håndterer både brugerinput og giver løbende feedback til brugeren. Derudover bliver data præsenteret på en overskuelig måde, og læreren har mulighed for at følge elevernes progression gennem et simpelt oversigtssystem. De non-funktionelle krav er også i høj grad opfyldt. Programmet er opdelt i tre lag, hvor logikken behandler data uafhængigt af brugerfladen. Der er anvendt objektorienteret programmering med mindst en klasse, og projektet er samlet som en kodebase, som er en offentligt repository. Derudover er der anvendt AI værktøjer som en del af udviklingsprocessen, hvilket har bidraget til både kode og forståelse af systemet. I forhold til milepælene blev der primært arbejdet med de to første, som også var de to vigtigste for at få en fungerende kerne af spillet. Det betyder at der er udviklet en singleplayer version med datalagring samt en oversigtsskærm til læreren. De senere milepæle, som multiplayer og login system, blev ikke færdiggjort. Det skyldes primært tidsmangel og ikke mindst gruppe arbejde.

Samlet set er projektet et spil som er opbygget i flere lag med fokus på struktur, genanvendelighed og brugeroplevelse. Selvom ikke alle planlagte funktioner er implementeret, opfylder løsningen de centrale krav. Man kan sige at det færdige produkt er et fungerende prototype der opfylder de centrale krav.

Bilag:

```
pygame.init()

#opretter vindue og sætter titel
pygame.display.set_caption("SpaceMath")
self.screen = pygame.display.set_mode((self.WIDTH, self.HEIGHT))
self.clock = pygame.time.Clock()
self.font = pygame.font.Font(None, 36)
self.score_font = pygame.font.SysFont("Consolas", 36)

base_path = os.path.dirname(__file__)
assets_path = os.path.join(base_path, "Stastik")

try:
    digital_font_path = os.path.join(assets_path, "digital-7.ttf")
    self.digital_font = pygame.font.Font(digital_font_path, 48)
except Exception:
    self.digital_font = pygame.font.SysFont("monospace", 48,
bold=True)

#database bruges til at gemme spilleners data
self.db = Database()
self.db.connect()

#opretter elve i database og gemmer id
self.student_id = self.db.add_student(student_name)

#hele spilrunde logic
self.game_session = GameSession()

self.answer_text: str = ""
self.feedback: str = ""
self.feedback_timer = 0
self.running = True
self.preview_active = False

self.x_offset = 0

path = os.path.join(assets_path, "spil.png")
self.background = pygame.image.load(path)
self.background = pygame.transform.scale(self.background, (self.WIDTH,
self.HEIGHT))

menu_path = os.path.join(assets_path, "menu.png")
self.menu_background = pygame.image.load(menu_path)
self.menu_background = pygame.transform.scale(self.menu_background,
(self.WIDTH, self.HEIGHT))
```

```
completion_path = os.path.join(assets_path, "slut.png")
try:
    self.completion_background = pygame.image.load(completion_path)
    self.completion_background =
pygame.transform.scale(self.completion_background, (self.WIDTH, self.HEIGHT))
except:
    self.completion_background = None

score_path = os.path.join(assets_path, "score.png")
try:
    self.score_background = pygame.image.load(score_path)
    self.score_background =
pygame.transform.scale(self.score_background, (self.WIDTH, self.HEIGHT))
except Exception:
    self.score_background = None

start_path = os.path.join(assets_path, "start.png")
try:
    self.start_background = pygame.image.load(start_path)
    self.start_background =
pygame.transform.scale(self.start_background, (self.WIDTH, self.HEIGHT))
except:
    self.start_background = None

#jeg bruger et state system til at styre spillets flow, så jeg kan have
#forskellige skærme
self.state = "start"
```

FIGUR 8 STATE SYSTEM OVERSIGT

```
if self.state == "start":
    self._render_start()
elif self.state == "menu":
    self._render_menu()
elif self.state == "scores":
    self._render_scores()
elif self.state == "completed":
    self._render_completed()
elif self.state == "teacher":
    self._render_teacher_overview()
else: # game
    self._render()
    if self.preview_active:
        self.blink_timer += 1
```

FIGUR 9 GAME LOOP STRUKTUR

```
def run(self) -> None:
    while self.running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                self.running = False

            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    self.running = False

            elif event.type == pygame.MOUSEBUTTONDOWN and event.button ==
1:
                if self.state == "start":
                    if self.elev_button.collidepoint(event.pos):
                        self.state = "menu"
                    elif self.teacher_button.collidepoint(event.pos):
                        self.state = "teacher"
                elif self.state == "menu":
                    score_text = self.font.render("Se din score her",
True, (255, 140, 0))
                    score_text_rect =
score_text.get_rect(center=(self.WIDTH // 2, self.score_button.centery))
                    if self.play_button.collidepoint(event.pos):
                        self.state = "game"
                        #reset for ny runde
                        self.game_session.reset()
                        self.answer_text = ""
                        self.feedback = ""
                        self.preview_active = False
                        self.typing_timer = 0
                        self.displayed_problem = ""
                        self.displayed_answer = ""
                        #starter preview for første problem
                        self._start_preview()
                    elif score_text_rect.collidepoint(event.pos):
                        self.state = "scores"
                elif self.state == "game":
                    self._handle_button_click(event.pos)
                elif self.state == "completed":
                    if self.back_button.collidepoint(event.pos):
                        self.state = "menu"
                elif self.state == "scores":
                    if self.scores_back_button.collidepoint(event.pos):
                        self.state = "menu"
                elif self.state == "teacher":
                    if self.teacher_back_button.collidepoint(event.pos):
                        self.state = "start"
```

```

        if self.state == "start":
            self._render_start()
        elif self.state == "menu":
            self._render_menu()
        elif self.state == "scores":
            self._render_scores()
        elif self.state == "completed":
            self._render_completed()
        elif self.state == "teacher":
            self._render_teacher_overview()
        else: # game
            self._render()
            if self.preview_active:
                self.blink_timer += 1

        self.clock.tick(30)

    self.db.close()
    pygame.quit()
    sys.exit()

```

FIGUR 10 RENDER-FUNKTION (GAME SCREEN)

```

def _render(self) -> None:
    self.screen.fill((0, 0, 0))
    self.screen.blit(self.background, (0, 0))

    current_problem = min(self.game_session.problems_solved + 1,
self.game_session.PROBLEMS_PER_ROUND)
    progress = f"{current_problem}/{self.game_session.PROBLEMS_PER_ROUND}"
    self._draw_text("SpaceMath", 80, 30, (255, 255, 0))
    self._draw_text(progress, 400, 30, (255, 255, 255))

    self._draw_text(f"Score: {self.game_session.score}", 720, 700, (0, 0,
0))

    if self.preview_active:
        self._draw_preview_button()
# viser problemmet efter 2 blinks
        if self.blink_timer >= 40 and not self.show_problem:
            self.show_problem = True
            self.typing_timer = 0
            self.displayed_problem = ""
            self.displayed_answer = ""
        if self.show_problem:
            self._update_typing_effect()

```

```

        self._draw_digital_text(self.displayed_problem, 80, 530, (255,
255, 255))
        self._draw_digital_text(self.displayed_answer, 320, 530, (255,
255, 255))
        elif self.feedback_timer > 0:
            self._draw_digital_text(self.feedback, 80, 530, (255, 255, 255))
        else:
            self._update_typing_effect()
            self._draw_digital_text(self.displayed_problem, 80, 530, (255,
255, 255))
            self._draw_digital_text(self.displayed_answer, 320, 530, (255,
255, 255))

        if self.feedback_timer > 0:
            self.feedback_timer -= 1
            if self.feedback_timer == 0:
                self.feedback = ""
#starter preview for næste problem
        if self.game_session.problems_solved <
self.game_session.PROBLEMS_PER_ROUND:
            self._start_preview()
        else:
            self.state = "completed"

        self._draw_buttons()
        self._draw_text("Click buttons to answer, Esc to quit", 200, 440,
(180, 180, 180))
        pygame.display.flip()

```

FIGUR 11 INPUT HÅNTERING

```

def _handle_button_click(self, pos: tuple[int, int]) -> None:
    for label, rect in self.button_rects.items():
        if rect.collidepoint(pos):
            if label == "OK":
                self._handle_submit()
            elif label == "Slet":
                self.answer_text = self.answer_text[:-1]
            else:
                self.answer_text += label
    return

```

FIGUR 12 PREVIEW FUNKTION

```
def _start_preview(self) -> None:
    next_problem = generate_problem()
    self.preview_operator = next_problem.operator
    self.preview_active = True
    self.blink_timer = 0
    self.typing_timer = 0 #reset typing effect
    self.game_session.current_problem = next_problem
    self.displayed_problem = ""
    self.displayed_answer = ""
    self.show_problem = False
```

FIGUR 13 TYPING EFFEKT

```
def _update_typing_effect(self) -> None:
    problem = self.game_session.current_problem
    full_problem = f"{problem.left} {problem.operator} {problem.right} ="

    if self.typing_timer % 5 == 0:
        if len(self.displayed_problem) < len(full_problem):
            self.displayed_problem = full_problem[:len(self.displayed_problem) + 1]

    self.displayed_answer = self.answer_text
    self.typing_timer += 1
```

FIGUR 14 EKSTRA FUNKTION / UI ELEMENT

```
if __name__ == "__main__":
    game = GameSession()

    print("=== SpaceMath terminal test ===")
    print("Skriv dit svar og tryk Enter. Skriv 'exit' for at stoppe.\n")

    while True:
        problem = game.get_current_problem()

        print(f"Opgave: {problem.left} {problem.operator} {problem.right} =
?")

        user_input = input("Dit svar: ")

        if user_input.lower() == "exit":
            print("Afslutter test...")
            break

        if not user_input.isdigit():
            print("Ugyldigt input - skriv et tal.\n")
            continue
```

```

    guess = int(user_input)

    result = game.submit_answer(guess)

    print(result["feedback"])
    print(f"Score: {result['score']} | Løst:
{result['problems_solved']}\n")

    if result["is_complete"]:
        print("Runde færdig!")
        print(f"Final score: {result['score']}/{game.PROBLEMS_PER_ROUND}")
        break

```

FIGUR 15 TERMINALBASERET TEST AF LOGIKLAG

Liste over figurer:

Figur 1 <i>Elevernes skræm undrer spillet</i>	4
Figur 2 <i>Lærernes skræm undrer spillet på evt. en stor skræm</i>	4
Figur 3 Rutediagram for præsentationslaget	8
Figur 4 Farvekodning i spillet (grøn = plus, rød = minus)	10
Figur 5 Input og feedback i spillet	10
Figur 6 Opstart af terminaltest	11
Figur 7 Feedback, score og nye opgaver i terminalen	12
Figur 8 State system oversigt.....	16
Figur 9 Game loop struktur.....	16
Figur 10 Render-funktion (game screen)	18
Figur 11 Input håndtering	19
Figur 12 Preview funktion	19
Figur 13 Typing effekt	20
Figur 14 Ekstra funktion / UI element	20
Figur 15 Terminalbaseret test af logiklag.....	21